

```

// js/alta-alumno.js
const IMGBB_API_KEY = '61a76cc12d06bd22948b4b5b76f5b45e';
const SVG_DEFAULT = "data:image/svg+xml;utf8,<svg
xmlns='http://www.w3.org/2000/svg' width='100' height='100' viewBox='0 0 24 24'
fill='none' stroke='%23a0aec0' stroke-width='1.5' stroke-linecap='round'
stroke-linejoin='round'><path d='M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4
4v2'></path><circle cx='12' cy='7' r='4'></circle></svg>";

let fotoBlobCapturada = null;
let videoStream = null;

// Agregar estas variables arriba en js/alta-alumno.js
let html5QrcodeScanner = null;

// Lógica para Abrir el Escáner QR
document.getElementById('btnEscanearQR').addEventListener('click', () => {
  const modalLector = document.getElementById('modalLectorQR');
  modalLector.style.display = 'flex';

  html5QrcodeScanner = new Html5Qrcode("reader");

  html5QrcodeScanner.start(
    { facingMode: "environment" }, // Usa la cámara trasera en móviles o la
    webcam en PC
    {
      fps: 10,
      qrbox: { width: 220, height: 220 }
    },
    (qrCodeMessage) => {
      // Éxito: Lectura detectada
      document.getElementById('txtCodigoQR').value = qrCodeMessage;
      cerrarLectorQR();
    },
    (errorMessage) => {
      // Búsqueda en progreso (no necesario hacer nada)
    }
  ).catch(err => {
    alert("Error al iniciar el lector de QR: " + err);
    cerrarLectorQR();
  });
});

document.getElementById('btnCerrarLectorQR').addEventListener('click',
cerrarLectorQR);

function cerrarLectorQR() {
  if (html5QrcodeScanner) {
    html5QrcodeScanner.stop().then(() => {
      html5QrcodeScanner.clear();
      document.getElementById('modalLectorQR').style.display = 'none';
    }).catch(err => console.error(err));
  } else {
    document.getElementById('modalLectorQR').style.display = 'none';
  }
}

```

```

}

document.addEventListener('DOMContentLoaded', () => {
  const imgPreview = document.getElementById('imgPreview');
  if (imgPreview) imgPreview.src = SVG_DEFAULT;

  // Cámara
  document.getElementById('btnAbrirCamara').addEventListener('click', async ()
=> {
    try {
      videoStream = await navigator.mediaDevices.getUserMedia({ video: { width:
400, height: 400 }, audio: false });
      document.getElementById('webcam').srcObject = videoStream;
      document.getElementById('modalCamara').style.display = 'flex';
    } catch (err) {
      alert("Error al abrir la cámara.");
    }
  });

  document.getElementById('btnCapturar').addEventListener('click', () => {
    const video = document.getElementById('webcam');
    const canvas = document.getElementById('canvasFoto');
    const context = canvas.getContext('2d');
    canvas.width = video.videoWidth || 300;
    canvas.height = video.videoHeight || 300;

    context.drawImage(video, 0, 0, canvas.width, canvas.height);
    canvas.toBlob((blob) => {
      fotoBlobCapturada = blob;
      imgPreview.src = URL.createObjectURL(blob);
      cerrarCamara();
    }, 'image/jpeg', 0.85);
  });

  document.getElementById('btnCerrarCamara').addEventListener('click',
cerrarCamara);

  function cerrarCamara() {
    if (videoStream) videoStream.getTracks().forEach(track => track.stop());
    document.getElementById('modalCamara').style.display = 'none';
  }

  // Submit de Alta
  document.getElementById('formAltaCajero').addEventListener('submit', async (e)
=> {
    e.preventDefault();
    const btn = document.getElementById('btnGuardar');
    btn.disabled = true;

    const codigoQR = document.getElementById('txtCodigoQR').value.trim();
    const dni = document.getElementById('txtDni').value.trim();
    const nombre = document.getElementById('txtNombre').value.trim();
    const curso = document.getElementById('txtCurso').value.trim();
    const pin = document.getElementById('txtPin').value.trim();
  });

```

```

if (pin.length !== 4 || isNaN(pin)) {
  mostrarError("El PIN debe ser de 4 dígitos numéricos.");
  btn.disabled = false;
  return;
}

let urlFoto = `https://api.dicebear.com/7.x/bottts/svg?seed=${dni}`;

try {
  if (fotoBlobCapturada) {
    btn.innerText = "Subiendo foto...";
    const formData = new FormData();
    formData.append('image', fotoBlobCapturada);

    const resImgBB = await
fetch(`https://api.imgbb.com/1/upload?key=${IMGBB_API_KEY}`, {
  method: 'POST',
  body: formData
});
    const dataImgBB = await resImgBB.json();
    if (dataImgBB.success) urlFoto = dataImgBB.data.url;
  }

  btn.innerText = "Guardando...";
  const { error } = await window._supabase
    .from('alumnos')
    .insert([
      {
        codigo_qr: codigoQR,
        dni: dni,
        nombre_apellido: nombre,
        curso: curso,
        foto_url: urlFoto,
        pin: pin,
        saldo: 0.00
      }
    ]);

  if (error) {
    if (error.code === '23505') {
      mostrarError("El DNI o el Código de Tarjeta ya existen en el
sistema.");
    } else {
      mostrarError("Error: " + error.message);
    }
  } else {
    alert(`¡Tarjeta asociadas con éxito a ${nombre}! Podés entregarle la
tarjeta.`);
    document.getElementById('formAltaCajero').reset();
    imgPreview.src = SVG_DEFAULT;
    fotoBlobCapturada = null;
    document.getElementById('txtCodigoQR').focus(); // Listo para el
siguiente

```

```

    }

    } catch (err) {
        console.error(err);
        mostrarError("Ocurrió un error inesperado.");
    } finally {
        btn.disabled = false;
        btn.innerText = "Registrar y Vincular Tarjeta";
    }
});

function mostrarError(txt) {
    const msgDiv = document.getElementById('msgAlta');
    msgDiv.innerText = txt;
    msgDiv.style.display = 'block';
}
});
// js/alumnos.js
document.addEventListener('DOMContentLoaded', async () => {
    // Verificar sesión
    const sessionData = sessionStorage.getItem('session');
    if (!sessionData) {
        window.location.href = 'index.html';
        return;
    }

    const session = JSON.parse(sessionData);
    document.getElementById('lblUsuario').innerText = `${session.nombre}`;

    document.getElementById('btnLogout').addEventListener('click', () => {
        sessionStorage.clear();
        window.location.href = 'index.html';
    });

    // Cargar lista de alumnos
    await cargarAlumnos();

    // Guardar Alumno
    document.getElementById('formAlumno').addEventListener('submit',
    registrarAlumno);

    // Filtro de búsqueda en tiempo real
    document.getElementById('txtBuscar').addEventListener('keyup', filtrarTabla);
});

let listaAlumnos = [];

async function cargarAlumnos() {
    const tbody = document.getElementById('tblAlumnos');
    try {
        const { data, error } = await window._supabase
            .from('alumnos')
            .select('*')
            .order('nombre_apellido', { ascending: true });
    }

```

```

    if (error) throw error;

    listaAlumnos = data || [];
    renderizarTabla(listaAlumnos);

} catch (err) {
    console.error("Error al cargar alumnos:", err);
    tbody.innerHTML = `|  |  |  |  |  |  |  |  |  |  |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| Error al cargar datos</td></tr>`; } }  function renderizarTabla(datos) {     const tbody = document.getElementById('tblAlumnos');     if (datos.length === 0) {         tbody.innerHTML = `|  |  |  |  |  | | --- | --- | --- | --- | --- | | No hay alumnos registrados.</td></tr>`;         return;     }      tbody.innerHTML = datos.map(a => `         <tr>             <td><b>${a.dni}</b></td>             <td>${a.nombre_apellido}</td>             <td>${a.curso}</td>             <td class="text-success"><b>$$${Number(a.saldo).toFixed(2)}</b></td>             <td>                 <button onclick="verQR('${a.dni}', '${a.nombre_apellido}')" style="background:#4a5568; color:white; border:none; padding:4px 8px; border-radius:4px; cursor:pointer;">  Ver QR                 </button>             </td>         </tr>     `).join(''); }  async function registrarAlumno(e) {     e.preventDefault();     const msgDiv = document.getElementById('msgAlumno');     msgDiv.style.display = 'none';      const dni = document.getElementById('dni').value.trim();     const nombre = document.getElementById('nombre').value.trim();     const curso = document.getElementById('curso').value.trim();     const pin = document.getElementById('pin').value.trim();      if (pin.length !== 4 || isNaN(pin)) {         mostrarMensaje("El PIN debe ser un número de 4 dígitos.", true);         return;     }      try { | | | | | | | | | |

```

```

const { error } = await window._supabase
  .from('alumnos')
  .insert([
    { dni, nombre_apellido: nombre, curso, pin, saldo: 0.00 }
  ]);

if (error) {
  if (error.code === '23505') { // Clave duplicada en postgres
    mostrarMensaje("El DNI ingresado ya se encuentra registrado.", true);
  } else {
    mostrarMensaje("Error al guardar: " + error.message, true);
  }
  return;
}

// Éxito
document.getElementById('formAlumno').reset();
await cargarAlumnos();
mostrarMensaje("Alumno registrado con éxito", false);

} catch (err) {
  console.error(err);
  mostrarMensaje("Error al procesar el registro.", true);
}
}

function filtrarTabla() {
  const texto = document.getElementById('txtBuscar').value.toLowerCase();
  const filtrados = listaAlumnos.filter(a =>
    a.dni.toLowerCase().includes(texto) ||
    a.nombre_apellido.toLowerCase().includes(texto) ||
    a.curso.toLowerCase().includes(texto)
  );
  renderizarTabla(filtrados);
}

// Generador de QR
let qrContainer = null;

function verQR(dni, nombre) {
  document.getElementById('qrNombre').innerText = nombre;
  document.getElementById('qrDni').innerText = `DNI: ${dni}`;

  const qrBox = document.getElementById('qrcode');
  qrBox.innerHTML = ''; // Limpiar previo

  // Generar QR que contiene simplemente el DNI del alumno
  qrContainer = new QRCode(qrBox, {
    text: dni,
    width: 160,
    height: 160,
    colorDark : "#1a365d",
    colorLight : "#ffffff",
    correctLevel : QRCode.CorrectLevel.H
  });
}

```

```

});

document.getElementById('modalQR').style.display = 'flex';
}

function cerrarModalQR() {
    document.getElementById('modalQR').style.display = 'none';
}

function imprimirQR() {
    window.print();
}

function mostrarMensaje(texto, esError) {
    const msgDiv = document.getElementById('msgAlumno');
    msgDiv.innerText = texto;
    msgDiv.style.color = esError ? '#e53e3e' : '#38a169';
    msgDiv.style.display = 'block';
}

// js/authGuard.js
(function checkAuth() {
    const sessionData = sessionStorage.getItem('session');
    const currentPath = window.location.pathname.split('/').pop();

    // Páginas públicas que no requieren login
    const publicPages = ['index.html', 'tablero-monitor.html', ''];

    if (publicPages.includes(currentPath)) {
        return; // Permitir acceso libre
    }

    // Si no hay sesión iniciada, mandar a login
    if (!sessionData) {
        window.location.href = 'index.html';
        return;
    }

    const session = JSON.parse(sessionData);

    // Restricción: Si un CAJERO intenta entrar a páginas exclusivas de ADMIN
    const adminOnlyPages = ['admin-dashboard.html', 'logs.html',
    'gestion-postnets.html'];

    if (session.rol !== 'ADMIN' && adminOnlyPages.includes(currentPath)) {
        alert("⊖ Acceso denegado: Esta sección requiere rol Administrador.");
        window.location.href = 'cajero-dashboard.html';
    }
})();

// js/cajero.js
let alumnoActual = null;
let html5QrcodeScanner = null;

document.addEventListener('DOMContentLoaded', () => {
    // 1. Validar Sesión del Operador

```

```

const sessionData = sessionStorage.getItem('session');
if (!sessionData) {
    window.location.href = 'index.html';
    return;
}

const session = JSON.parse(sessionData);
document.getElementById('lblUsuario').innerText = `${session.nombre}
(${session.rol})`;

// Habilitar / Ocultar la sección de extracción según los permisos habilitados
en usuarios_banco
const secExtraccion = document.getElementById('secExtraccion');
if (secExtraccion) {
    if (session.puede_retirar === true || session.rol === 'ADMIN') {
        secExtraccion.style.display = 'block';
    } else {
        secExtraccion.style.display = 'none';
    }
}

// Cerrar Sesión
document.getElementById('btnLogout').addEventListener('click', () => {
    sessionStorage.clear();
    window.location.href = 'index.html';
});

// 2. Eventos de Búsqueda
document.getElementById('btnBuscar').addEventListener('click', () => {
    const query = document.getElementById('txtBuscarDni').value.trim();
    if (query) buscarAlumno(query);
});

document.getElementById('txtBuscarDni').addEventListener('keypress', (e) => {
    if (e.key === 'Enter') {
        const query = document.getElementById('txtBuscarDni').value.trim();
        if (query) buscarAlumno(query);
    }
});

// Escáner QR por cámara
document.getElementById('btnEscanearQR').addEventListener('click',
abrirLectorQR);
document.getElementById('btnCerrarLectorQR').addEventListener('click',
cerrarLectorQR);

// Submit Formularios
document.getElementById('formRecarga').addEventListener('submit',
procesarRecarga);
document.getElementById('formResetPin').addEventListener('submit',
procesarResetPin);

const formExtraccion = document.getElementById('formExtraccion');
if (formExtraccion) {

```



```

        formExtraccion.addEventListener('submit', procesarExtraccion);
    }
});

// BÚSQUEDA DE ALUMNO (Por DNI o por Código QR)
async function buscarAlumno(criterio) {
    try {
        const { data, error } = await window._supabase
            .from('alumnos')
            .select('*')
            .or(`dni.eq.${criterio},codigo_qr.eq.${criterio}`)
            .single();

        if (error || !data) {
            alert("Alumno no encontrado. Verificá el DNI o el QR de la tarjeta.");
            document.getElementById('panelAlumno').style.display = 'none';
            alumnoActual = null;
            return;
        }

        alumnoActual = data;

        // Actualizar Panel Visual
        document.getElementById('lblNombre').innerText = data.nombre_apellido;
        document.getElementById('lblCurso').innerText = data.curso || 'Sin curso';
        document.getElementById('lblDni').innerText = data.dni;
        document.getElementById('lblQR').innerText = data.codigo_qr || 'Sin QR registrado';
        document.getElementById('lblSaldo').innerText = `$$${Number(data.saldo || 0).toFixed(2)}`;

        const fotoDefault = "data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='100' height='100' viewBox='0 0 24 24' fill='none' stroke='%23a0aec0'><circle cx='12' cy='7' r='4'/></svg>";
        document.getElementById('imgAlumno').src = data.foto_url || fotoDefault;

        document.getElementById('panelAlumno').style.display = 'grid';

    } catch (err) {
        console.error(err);
        alert("Error al realizar la consulta en la base de datos.");
    }
}

// 1. PROCESAR RECARGA DE SALDO
async function procesarRecarga(e) {
    e.preventDefault();
    if (!alumnoActual) return;

    const monto = parseFloat(document.getElementById('montoRecarga').value);
    if (isNaN(monto) || monto <= 0) {
        alert("Ingresá un monto válido.");
        return;
    }
}

```

```

const session = JSON.parse(sessionStorage.getItem('session'));
const saldoActual = Number(alumnoActual.saldo || 0);
const nuevoSaldo = saldoActual + monto;

try {
  // Actualizar Saldo filtrando por DNI
  const { error: errUpdate } = await window._supabase
    .from('alumnos')
    .update({ saldo: nuevoSaldo })
    .eq('dni', alumnoActual.dni);

  if (errUpdate) throw errUpdate;

  // Registrar Transacción
  await window._supabase.from('transacciones').insert([
    {
      alumno_dni: alumnoActual.dni,
      usuario_banco_id: session.id || null,
      monto: monto,
      tipo: 'RECARGA',
      estado: 'OK'
    }
  ]);

  // Log de Auditoría
  await registrarLog(
    session.usuario,
    session.rol,
    'RECARGA_SALDO',
    `Recarga de ${monto.toFixed(2)} a ${alumnoActual.nombre_apellido}. Saldo anterior: ${saldoActual.toFixed(2)} | Nuevo: ${nuevoSaldo.toFixed(2)}`,
    alumnoActual.dni
  );

  alert(`¡Carga exitosa! Nuevo saldo de ${alumnoActual.nombre_apellido}: ${nuevoSaldo.toFixed(2)}`);

  document.getElementById('formRecarga').reset();
  buscarAlumno(alumnoActual.dni);

} catch (err) {
  console.error(err);
  alert("Error al procesar la carga: " + err.message);
}

// 2. PROCESAR EXTRACCIÓN DE SALDO (Requiere PIN)
async function procesarExtraccion(e) {
  e.preventDefault();
  if (!alumnoActual) return;

  const monto = parseFloat(document.getElementById('montoExtraccion').value);
  const pinIngresado = document.getElementById('pinExtraccion').value.trim();

  if (isNaN(monto) || monto <= 0) {

```

```

    alert("Ingresá un monto válido para extraer.");
    return;
}

const saldoActual = Number(alumnoActual.saldo || 0);

if (monto > saldoActual) {
    alert(`Fondos insuficientes. El alumno solo dispone de
    ${saldoActual.toFixed(2)}`);
    return;
}

// Validar PIN de Autorización del Alumno
if (pinIngresado !== alumnoActual.pin) {
    alert("⊖ PIN incorrecto. El alumno debe ingresar su clave secreta de 4
    dígitos.");
    document.getElementById('pinExtraccion').value = '';
    return;
}

const session = JSON.parse(sessionStorage.getItem('session'));
const nuevoSaldo = saldoActual - monto;

try {
    // Actualizar Saldo en BD
    const { error: errUpdate } = await window._supabase
        .from('alumnos')
        .update({ saldo: nuevoSaldo })
        .eq('dni', alumnoActual.dni);

    if (errUpdate) throw errUpdate;

    // Registrar Transacción
    await window._supabase.from('transacciones').insert([
        {
            alumno_dni: alumnoActual.dni,
            usuario_banco_id: session.id || null,
            monto: monto,
            tipo: 'EXTRACCION',
            estado: 'OK'
        }
    ]);

    // Log de Auditoría
    await registrarLog(
        session.usuario,
        session.rol,
        'EXTRACCION_SALDO',
        `Extracción de ${monto.toFixed(2)} entregada a
        ${alumnoActual.nombre_apellido}. Saldo anterior: ${saldoActual.toFixed(2)} |
        Nuevo: ${nuevoSaldo.toFixed(2)}`,
        alumnoActual.dni
    );

    alert(`¡Extracción autorizada! Entregar ${monto.toFixed(2)} en efectivo a
    ${alumnoActual.nombre_apellido}. \nNuevo saldo: ${nuevoSaldo.toFixed(2)}`);
}

```

```

        document.getElementById('formExtraccion').reset();
        buscarAlumno(alumnoActual.dni);

    } catch (err) {
        console.error(err);
        alert("Error al procesar el retiro: " + err.message);
    }
}

// 3. RESTAURAR / RESETEAR PIN
async function procesarResetPin(e) {
    e.preventDefault();
    if (!alumnoActual) return;

    const nuevoPin = document.getElementById('nuevoPin').value.trim();
    if (nuevoPin.length !== 4 || isNaN(nuevoPin)) {
        alert("El PIN debe ser una clave numérica de 4 dígitos.");
        return;
    }

    const session = JSON.parse(sessionStorage.getItem('session'));

    try {
        const { error } = await window._supabase
            .from('alumnos')
            .update({ pin: nuevoPin })
            .eq('dni', alumnoActual.dni);

        if (error) throw error;

        // Log de Auditoría
        await registrarLog(
            session.usuario,
            session.rol,
            'RESET_PIN',
            `Restauración de PIN realizada para el alumno
            ${alumnoActual.nombre_apellido}`,
            alumnoActual.dni
        );

        alert("¡PIN actualizado con éxito!");
        document.getElementById('formResetPin').reset();

    } catch (err) {
        console.error(err);
        alert("Error al restaurar el PIN: " + err.message);
    }
}

// FUNCION GENERAL DE AUDITORÍA (logs_sistema)
async function registrarLog(usuario, rol, accion, detalle, alumnoDni = null) {
    try {
        await window._supabase.from('logs_sistema').insert([

```

```

        usuario_operador: usuario,
        rol_operador: rol,
        accion: accion,
        detalle: detalle,
        alumno_dni: alumnoDni
    ]]);
} catch (err) {
    console.error("Error al registrar en logs_sistema:", err);
}
}

// LECTOR QR CÁMARA
function abrirLectorQR() {
    document.getElementById('modalLectorQR').style.display = 'flex';
    html5QrcodeScanner = new Html5Qrcode("reader");
    html5QrcodeScanner.start(
        { facingMode: "environment" },
        { fps: 10, qrbox: { width: 220, height: 220 } },
        (qrMessage) => {
            document.getElementById('txtBuscarDni').value = qrMessage;
            cerrarLectorQR();
            buscarAlumno(qrMessage);
        },
        () => {}
    );
}

function cerrarLectorQR() {
    if (html5QrcodeScanner) {
        html5QrcodeScanner.stop().then(() => {
            html5QrcodeScanner.clear();
            document.getElementById('modalLectorQR').style.display = 'none';
        }).catch(() => document.getElementById('modalLectorQR').style.display =
'none');
    } else {
        document.getElementById('modalLectorQR').style.display = 'none';
    }
}

// js/dashboard.js
document.addEventListener('DOMContentLoaded', async () => {
    // 1. Control de Seguridad: Verificar Sesión Activa
    const sessionData = sessionStorage.getItem('session');
    if (!sessionData) {
        window.location.href = 'index.html';
        return;
    }

    const session = JSON.parse(sessionData);
    document.getElementById('lblUsuario').innerText = `${session.nombre}
(${session.rol})`;

    // Botón de Cerrar Sesión
    document.getElementById('btnLogout').addEventListener('click', () => {
        sessionStorage.clear();
    });

```

```

    window.location.href = 'index.html';
  });

  // 2. Cargar Métricas y Tablas
  await cargarMetricas();
  await cargarUltimasTransacciones();
});

async function cargarMetricas() {
  try {
    // A. Consultar total de Alumnos y Saldo Circulante
    const { data: alumnos, error: errAlumnos } = await window._supabase
      .from('alumnos')
      .select('saldo');

    if (!errAlumnos && alumnos) {
      document.getElementById('kpiAlumnos').innerText = alumnos.length;
      const saldoTotal = alumnos.reduce((sum, item) => sum + Number(item.saldo
|| 0), 0);
      document.getElementById('kpiSaldoCirculante').innerText =
`$${saldoTotal.toFixed(2)}`;
    }

    // B. Consultar total de Posnets/Stands activos
    const { count: countPosnets, error: errPosnets } = await window._supabase
      .from('postnets')
      .select('*', { count: 'exact', head: true })
      .eq('activo', true);

    if (!errPosnets) {
      document.getElementById('kpiPosnets').innerText = countPosnets || 0;
    }

    // C. Consultar Transacciones para sumar Ingresos (RECARGA) y Egresos (COBRO)
    const { data: transacciones, error: errTrans } = await window._supabase
      .from('transacciones')
      .select('monto, tipo, estado');

    if (!errTrans && transacciones) {
      let totalIngresos = 0;
      let totalEgresos = 0;

      transacciones.forEach(t => {
        if (t.estado === 'OK') {
          if (t.tipo === 'RECARGA') totalIngresos += Number(t.monto);
          if (t.tipo === 'COBRO') totalEgresos += Number(t.monto);
        }
      });

      document.getElementById('kpiIngresos').innerText =
`$${totalIngresos.toFixed(2)}`;
      document.getElementById('kpiEgresos').innerText =
`$${totalEgresos.toFixed(2)}`;
    }
  }
}

```

```

    }

    } catch (e) {
        console.error("Error al obtener métricas:", e);
    }
}

async function cargarUltimasTransacciones() {
    const tbody = document.getElementById('tblTransacciones');

    try {
        const { data, error } = await window._supabase
            .from('transacciones')
            .select(`
                id,
                alumno_dni,
                monto,
                tipo,
                estado,
                fecha_hora,
                postnets ( nombre_stand )
            `)
            .order('fecha_hora', { ascending: false })
            .limit(10);

        if (error) throw error;

        if (!data || data.length === 0) {
            tbody.innerHTML = `<tr><td colspan="6" class="text-center">No hay
movimientos registrados aún.</td></tr>`;
            return;
        }

        tbody.innerHTML = data.map(t => {
            const fecha = new Date(t.fecha_hora).toLocaleString('es-AR');
            const badgeClass = t.tipo === 'RECARGA' ? 'badge-recarga' : 'badge-cobro';
            const standNombre = t.postnets ? t.postnets.nombre_stand : 'Caja
Principal';

            return `
                <tr>
                    <td>${fecha}</td>
                    <td><b>${t.alumno_dni}</b></td>
                    <td><span class="badge ${badgeClass}">${t.tipo}</span></td>
                    <td>${standNombre}</td>
                    <td><b>${Number(t.monto).toFixed(2)}</b></td>
                    <td>${t.estado}</td>
                </tr>
            `;
        }).join('');

    } catch (e) {
        console.error("Error al cargar transacciones:", e);
        tbody.innerHTML = `<tr><td colspan="6" class="text-center text-danger">Error

```

```

al cargar historial.</td></tr>`;
}
}
// js/registro.js
// Clave API de ImgBB (Reemplazá con tu clave de api.imgbb.com)
const IMGBB_API_KEY = '61a76cc12d06bd22948b4b5b76f5b45e';

// SVG por defecto para que NUNCA se rompa la vista previa
const SVG_DEFAULT = "data:image/svg+xml;utf8,<svg
xmlns='http://www.w3.org/2000/svg' width='110' height='110' viewBox='0 0 24 24'
fill='none' stroke='%23a0aec0' stroke-width='1.5' stroke-linecap='round'
stroke-linejoin='round'><path d='M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4
4v2'></path><circle cx='12' cy='7' r='4'></circle></svg>";

let fotoBlobCapturada = null;
let videoStream = null;

document.addEventListener('DOMContentLoaded', () => {
  const modalCamara = document.getElementById('modalCamara');
  const video = document.getElementById('webcam');
  const canvas = document.getElementById('canvasFoto');
  const imgPreview = document.getElementById('imgPreview');

  // Asegurar que la vista previa inicie con el SVG válido
  if (imgPreview) imgPreview.src = SVG_DEFAULT;

  // 1. Abrir modal y activar webcam
  document.getElementById('btnAbrirCamara').addEventListener('click', async ()
=> {
    try {
      videoStream = await navigator.mediaDevices.getUserMedia({ video: { width:
400, height: 400 }, audio: false });
      video.srcObject = videoStream;
      modalCamara.style.display = 'flex';
    } catch (err) {
      alert("No se pudo acceder a la cámara. Comprabá los permisos del
navegador.");
      console.error(err);
    }
  });

  // 2. Capturar foto desde el video hacia el canvas
  document.getElementById('btnCapturar').addEventListener('click', () => {
    const context = canvas.getContext('2d');
    canvas.width = video.videoWidth || 300;
    canvas.height = video.videoHeight || 300;

    context.drawImage(video, 0, 0, canvas.width, canvas.height);

    canvas.toBlob((blob) => {
      fotoBlobCapturada = blob;
      imgPreview.src = URL.createObjectURL(blob); // Muestra la foto tomada
      cerrarCamara();
    }, 'image/jpeg', 0.85);
  });

```



```

});

// 3. Cerrar cámara
document.getElementById('btnCerrarCamara').addEventListener('click',
cerrarCamara);

function cerrarCamara() {
  if (videoStream) {
    videoStream.getTracks().forEach(track => track.stop());
  }
  modalCamara.style.display = 'none';
}

// 4. Procesar Registro
document.getElementById('formRegistroAlumno').addEventListener('submit', async
(e) => {
  e.preventDefault();
  const btnGuardar = document.getElementById('btnGuardar');
  btnGuardar.disabled = true;
  btnGuardar.innerText = "Procesando...";

  const dni = document.getElementById('regDni').value.trim();
  const nombre = document.getElementById('regNombre').value.trim();
  const curso = document.getElementById('regCurso').value.trim();
  const pin = document.getElementById('regPin').value.trim();

  if (pin.length !== 4 || isNaN(pin)) {
    mostrarError("El PIN debe ser un número de 4 dígitos.");
    btnGuardar.disabled = false;
    btnGuardar.innerText = "Crear Mi Cuenta";
    return;
  }

  let urlFotoFinal = `https://api.dicebear.com/7.x/bottts/svg?seed=${dni}`;

  try {
    if (fotoBlobCapturada) {
      btnGuardar.innerText = "Subiendo foto...";

      const formData = new FormData();
      formData.append('image', fotoBlobCapturada);

      const resImgBB = await
fetch(`https://api.imgbb.com/1/upload?key=${IMGBB_API_KEY}`, {
  method: 'POST',
  body: formData
});

      const dataImgBB = await resImgBB.json();

      if (dataImgBB.success) {
        urlFotoFinal = dataImgBB.data.url;
      }
    }
  }

```

```

btnGuardar.innerText = "Registrando alumno...";
const { error } = await window._supabase
  .from('alumnos')
  .insert([
    {
      dni: dni,
      nombre_apellido: nombre,
      curso: curso,
      foto_url: urlFotoFinal,
      pin: pin,
      saldo: 0.00
    }
  ]);

if (error) {
  if (error.code === '23505') {
    mostrarError("Este DNI ya se encuentra registrado.");
  } else {
    mostrarError("Error al registrarse: " + error.message);
  }
} else {
  alert("¡Cuenta creada exitosamente! Podés dirigirte al cajero a cargar saldo.");

  // Reset de campos y RESTAURACIÓN DE VISTA PREVIA
  document.getElementById('formRegistroAlumno').reset();
  imgPreview.src = SVG_DEFAULT; // <-- AQUÍ SE RELLENA EL SVG NUEVAMENTE
  fotoBlobCapturada = null;
}

} catch (err) {
  console.error(err);
  mostrarError("Ocurrió un error inesperado.");
} finally {
  btnGuardar.disabled = false;
  btnGuardar.innerText = "Crear Mi Cuenta";
}
});

function mostrarError(txt) {
  const msgDiv = document.getElementById('msgRegistro');
  msgDiv.innerText = txt;
  msgDiv.style.display = 'block';
}
});

```